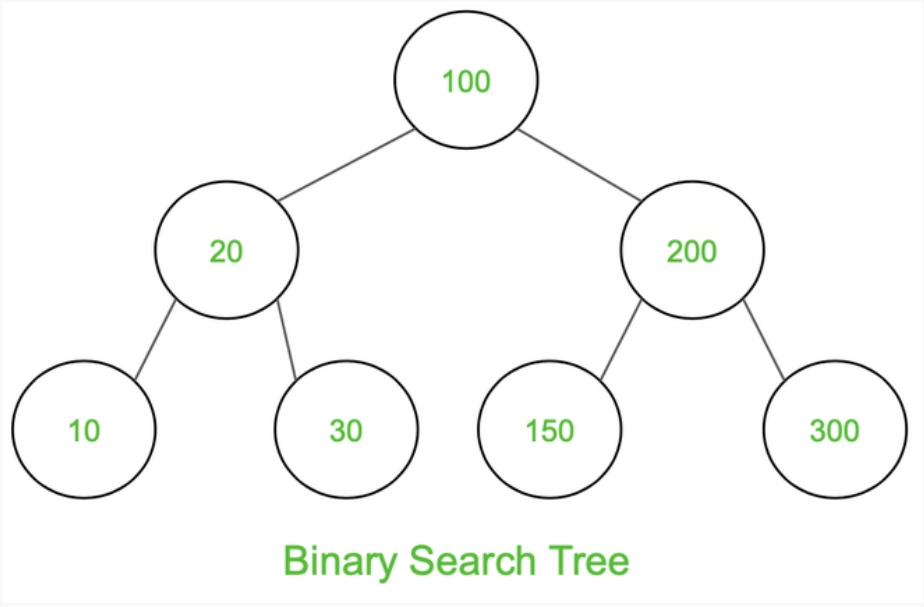
Binary Search Tree (BST) Traversals – Inorder, Preorder, Post Order

Last Updated : 21 Aug, 2024

Given a Binary Search Tree, The task is to print the elements in inorder, preorder, and postorder traversal of the Binary Search Tree.

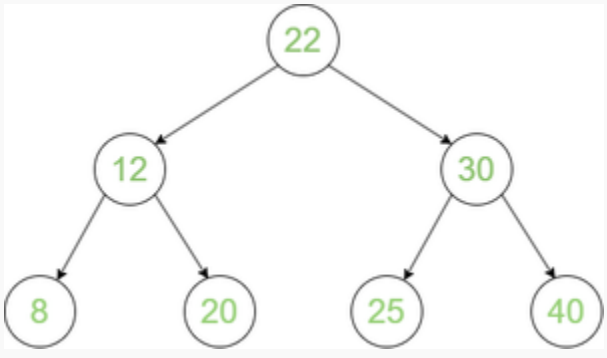
Input:



A Binary Search Tree

Output:
Inorder Traversal: 10 20 30 100 150 200 300
Preorder Traversal: 100 20 10 30 200 150 300
Postorder Traversal: 10 30 20 150 300 200 100

Input:



Binary Search Tree

Output:
Inorder Traversal: 8 12 20 22 25 30 40
Preorder Traversal: 22 12 8 20 30 25 40
Postorder Traversal: 8 20 12 25 40 30 22

Inorder Traversal:

Below is the idea to solve the problem:

> *At first traverse **left subtree** then visit the **root** and then traverse the **right subtree**.*

Follow the below steps to implement the idea:

- Traverse left subtree
- Visit the root and print the data.
- Traverse the right subtree

The inorder traversal of the BST gives the values of the nodes in sorted order. To get the decreasing order visit the right, root, and left subtree.

Below is the implementation of the inorder traversal.

C++ Java Python C# JavaScript

```cpp
// C++ code to implement the approach

#include <bits/stdc++.h>
using namespace std;

// Class describing a node of tree
class Node {
public:
    int data;
    Node* left;
    Node* right;
    Node(int v)
    {
        this->data = v;
        this->left = this->right = NULL;
    }
};

// Inorder Traversal
void printInorder(Node* node)
{
    if (node == NULL)
```

```
        return;

    // Traverse left subtree
    printInorder(node->left);

    // Visit node
    cout << node->data << " ";

    // Traverse right subtree
    printInorder(node->right);
}

// Driver code
int main()
{
    // Build the tree
    Node* root = new Node(100);
    root->left = new Node(20);
    root->right = new Node(200);
    root->left->left = new Node(10);
    root->left->right = new Node(30);
    root->right->left = new Node(150);
    root->right->right = new Node(300);

    // Function call
    cout << "Inorder Traversal: ";
    printInorder(root);
    return 0;
}
```

Output

```
Inorder Traversal: 10 20 30 100 150 200 300
```

Time complexity: O(N), Where N is the number of nodes.

Auxiliary Space: O(h), Where h is the height of tree

Preorder Traversal:

Below is the idea to solve the problem:

*At first visit the **root** then traverse **left subtree** and then traverse the **right subtree**.*

Follow the below steps to implement the idea:

- Visit the root and print the data.
- Traverse left subtree
- Traverse the right subtree

Below is the implementation of the preorder traversal.

C++ Java Python C# JavaScript

```cpp
// C++ code to implement the approach

#include <bits/stdc++.h>
using namespace std;

// Class describing a node of tree
class Node {
public:
    int data;
    Node* left;
    Node* right;
    Node(int v)
    {
        this->data = v;
        this->left = this->right = NULL;
    }
};

// Preorder Traversal
void printPreOrder(Node* node)
{
    if (node == NULL)
        return;

    // Visit Node
    cout << node->data << " ";

    // Traverse left subtree
```

```
    printPreOrder(node->left);

    // Traverse right subtree
    printPreOrder(node->right);
}

// Driver code
int main()
{
    // Build the tree
    Node* root = new Node(100);
    root->left = new Node(20);
    root->right = new Node(200);
    root->left->left = new Node(10);
    root->left->right = new Node(30);
    root->right->left = new Node(150);
    root->right->right = new Node(300);

    // Function call
    cout << "Preorder Traversal: ";
    printPreOrder(root);
    return 0;
}
```

Output

```
Preorder Traversal: 100 20 10 30 200 150 300
```

Time complexity: O(N), Where N is the number of nodes.

Auxiliary Space: O(H), Where H is the height of the tree

Postorder Traversal:

Below is the idea to solve the problem:

> *At first traverse **left subtree** then traverse the **right subtree** and then visit the **root**.*

Follow the below steps to implement the idea:

- Traverse left subtree

- Traverse the right subtree
- Visit the root and print the data.

Below is the implementation of the postorder traversal:

C++ Java Python C# JavaScript

```cpp
// C++ code to implement the approach

#include <bits/stdc++.h>
using namespace std;

// Class to define structure of a node
class Node {
public:
    int data;
    Node* left;
    Node* right;
    Node(int v)
    {
        this->data = v;
        this->left = this->right = NULL;
    }
};

// PostOrder Traversal
void printPostOrder(Node* node)
{
    if (node == NULL)
        return;

    // Traverse left subtree
    printPostOrder(node->left);

    // Traverse right subtree
    printPostOrder(node->right);

    // Visit node
    cout << node->data << " ";
}
```

```
// Driver code
int main()
{
    Node* root = new Node(100);
    root->left = new Node(20);
    root->right = new Node(200);
    root->left->left = new Node(10);
    root->left->right = new Node(30);
    root->right->left = new Node(150);
    root->right->right = new Node(300);

    // Function call
    cout << "PostOrder Traversal: ";
    printPostOrder(root);
    cout << "\n";

    return 0;
}
```

Output

```
PostOrder Traversal: 10 30 20 150 300 200 100
```

Time complexity: O(N), Where N is the number of nodes.

Auxiliary Space: O(H), Where H is the height of the tree

Next Article

Balance a Binary Search Tree

Similar Reads

Binary Search Tree

A Binary Search Tree (or BST) is a data structure used in computer science for organizing and storing data in a sorted manner. Each node in a Binary Search Tree has at most two children, a left child and a right...

3 min read

Introduction to Binary Search Tree

Binary Search Tree is a data structure used in computer science for organizing and storing data in a sorted manner. Binary search tree follows all properties of binary tree and for every nodes, its left subtree...

3 min read

Applications of BST

Binary Search Tree (BST) is a data structure that is commonly used to implement efficient searching, insertion, and deletion operations along with maintaining sorted sequence of data. Please remember th...

3 min read

Applications, Advantages and Disadvantages of Binary Search Tree

A Binary Search Tree (BST) is a data structure used to storing data in a sorted manner. Each node in a Binary Search Tree has at most two children, a left child and a right child, with the left child containing...

2 min read

Insertion in Binary Search Tree (BST)

Given a BST, the task is to insert a new node in this BST.Example: How to Insert a value in a Binary Search Tree:A new key is always inserted at the leaf by maintaining the property of the binary search...

15 min read

Searching in Binary Search Tree (BST)

Given a BST, the task is to search a node in this BST. For searching a value in BST, consider it as a sorted array. Now we can easily perform search operation in BST using Binary Search Algorithm. Input: Root of...

7 min read

Deletion in Binary Search Tree (BST)

Given a BST, the task is to delete a node in this BST, which can be broken down into 3 scenarios:Case 1. Delete a Leaf Node in BSTDeletion in BSTCase 2. Delete a Node with Single Child in BSTDeleting a...

10 min read

Binary Search Tree (BST) Traversals â€“ Inorder, Preorder, Post Order

Given a Binary Search Tree, The task is to print the elements in inorder, preorder, and postorder traversal of the Binary Search Tree.Â Input:Â A Binary Search TreeOutput:Â Inorder Traversal: 10 20 30 100 150...

10 min read

Balance a Binary Search Tree

Given a BST (Binary Search Tree) that may be unbalanced, the task is to convert it into a balanced BST that has the minimum possible height.Examples: Input: Output: Explanation: The above unbalanced BST...

10 min read

Self-Balancing Binary Search Trees

Self-Balancing Binary Search Trees are height-balanced binary search trees that automatically keep the height as small as possible when insertion and deletion operations are performed on the tree. The heigh...

4 min read